

✎ Klasifikasi Objek Langit (Bintang, Galaksi, Quasar)

Komparasi Naïve Bayes, KNN, dan Decision Tree **dengan SMOTE**

Dataset: SDSS (Sloan Digital Sky Survey)

Catatan: Notebook ini merupakan versi dengan penyeimbangan kelas menggunakan **SMOTE** (Synthetic Minority Over-sampling Technique). Distribusi kelas asli: GALAXY $\approx$ 59%, STAR $\approx$ 21%, QSO $\approx$ 18%. SMOTE diterapkan **hanya pada data latih** untuk menghindari data leakage ke data uji. Perbandingan dengan versi tanpa SMOTE dirangkum di bagian akhir.

✎ 0. Persiapan Environment dan Import Library

```
# Install imbalanced-learn jika belum tersedia
!pip install imbalanced-learn -q
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time

from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)
from imblearn.over_sampling import SMOTE

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 110
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
print("✅ Library berhasil diimport")
```

✅ Library berhasil diimport

✎ 1. Load Dataset

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
DATA_PATH = '/content/drive/MyDrive/Dataset/star_classification.csv'

df = pd.read_csv(DATA_PATH)

assert df.shape[0] == 100000, f"⚠️ Jumlah baris tidak sesuai! Hanya terdapat {df.shape[0]}."
assert df.isnull().sum().sum() == 0, "⚠️ Ada missing value yang tidak diharapkan!"
print("✅ Data terverifikasi lengkap:", df.shape)
```

✅ Data terverifikasi lengkap: (100000, 18)

✎ 2. Eksplorasi Data (EDA)

Tahap ini memahami struktur data, distribusi kelas **sebelum** SMOTE diterapkan.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 18 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   obj_ID      100000 non-null  float64
1   alpha       100000 non-null  float64
2   delta       100000 non-null  float64
3   u           100000 non-null  float64
4   g           100000 non-null  float64
5   r           100000 non-null  float64
6   i           100000 non-null  float64
7   z           100000 non-null  float64
8   run_ID      100000 non-null  int64
9   rerun_ID    100000 non-null  int64
10  cam_col     100000 non-null  int64
11  field_ID    100000 non-null  int64
12  spec_obj_ID 100000 non-null  float64
13  class       100000 non-null  object
14  redshift    100000 non-null  float64
15  plate       100000 non-null  int64
16  MJD         100000 non-null  int64
17  fiber_ID    100000 non-null  int64
dtypes: float64(10), int64(7), object(1)
memory usage: 13.7+ MB
```

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
obj_ID	100000.0	1.237665e+18	8.438560e+12	1.237646e+18	1.237659e+18	1.237663e+18	1.237668e+18	1.237681e+18
alpha	100000.0	1.776291e+02	9.650224e+01	5.527828e-03	1.275182e+02	1.809007e+02	2.338950e+02	3.599998e+02
delta	100000.0	2.413530e+01	1.964467e+01	-1.878533e+01	5.146771e+00	2.364592e+01	3.990155e+01	8.300052e+01
u	100000.0	2.198047e+01	3.176929e+01	-9.999000e+03	2.035235e+01	2.217914e+01	2.368744e+01	3.278139e+01
g	100000.0	2.053139e+01	3.175029e+01	-9.999000e+03	1.896523e+01	2.109983e+01	2.212377e+01	3.160224e+01
r	100000.0	1.964576e+01	1.854760e+00	9.822070e+00	1.813583e+01	2.012529e+01	2.104478e+01	2.957186e+01
i	100000.0	1.908485e+01	1.757895e+00	9.469903e+00	1.773228e+01	1.940514e+01	2.039650e+01	3.214147e+01
z	100000.0	1.866881e+01	3.172815e+01	-9.999000e+03	1.746068e+01	1.900460e+01	1.992112e+01	2.938374e+01
run_ID	100000.0	4.481366e+03	1.964765e+03	1.090000e+02	3.187000e+03	4.188000e+03	5.326000e+03	8.162000e+03
rerun_ID	100000.0	3.010000e+02	0.000000e+00	3.010000e+02	3.010000e+02	3.010000e+02	3.010000e+02	3.010000e+02
cam_col	100000.0	3.511610e+00	1.586912e+00	1.000000e+00	2.000000e+00	4.000000e+00	5.000000e+00	6.000000e+00
field_ID	100000.0	1.861305e+02	1.490111e+02	1.100000e+01	8.200000e+01	1.460000e+02	2.410000e+02	9.890000e+02
spec_obj_ID	100000.0	5.783882e+18	3.324016e+18	2.995191e+17	2.844138e+18	5.614883e+18	8.332144e+18	1.412694e+19
redshift	100000.0	5.766608e-01	7.307073e-01	-9.970667e-03	5.451684e-02	4.241733e-01	7.041543e-01	7.011245e+00
plate	100000.0	5.137010e+03	2.952303e+03	2.660000e+02	2.526000e+03	4.987000e+03	7.400250e+03	1.254700e+04
MJD	100000.0	5.558865e+04	1.808484e+03	5.160800e+04	5.423400e+04	5.586850e+04	5.677700e+04	5.893200e+04
fiber_ID	100000.0	4.493127e+02	2.724984e+02	1.000000e+00	2.210000e+02	4.330000e+02	6.450000e+02	1.000000e+03

```
# Distribusi kelas asli (sebelum SMOTE)
plt.figure(figsize=(6,4))
ax = sns.countplot(data=df, x="class", order=df["class"].value_counts().index, palette="viridis")
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x()+p.get_width()/2, p.get_height()),
               ha='center', va='bottom')
plt.title("Distribusi Kelas Objek Langit (SEBELUM SMOTE - Distribusi Alami)")
plt.xlabel("Kelas")
plt.ylabel("Jumlah Objek")
plt.show()

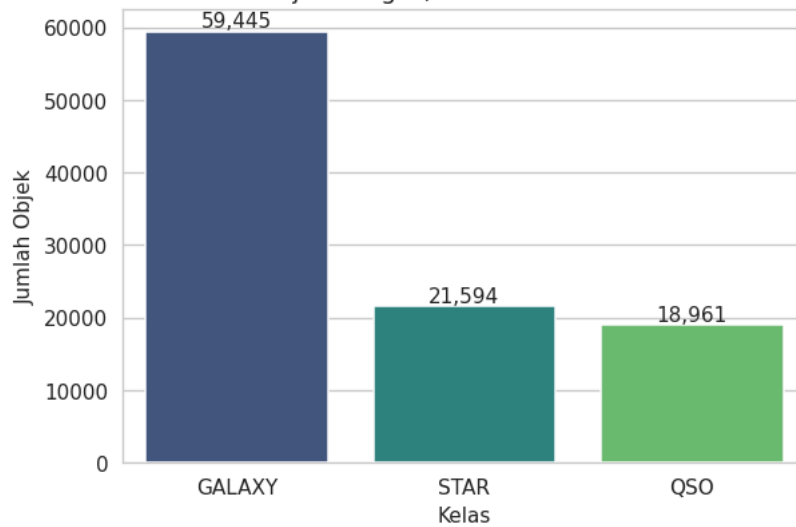
print(df["class"].value_counts())
print("\nPersentase:")
print(df["class"].value_counts(normalize=True).round(3) * 100, "%")
```

/tmp/ipykernel_865/453166398.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to

```
ax = sns.countplot(data=df, x="class", order=df["class"].value_counts().index, palette="viridis")
```

Distribusi Kelas Objek Langit (SEBELUM SMOTE - Distribusi Alami)



```
class
GALAXY    59445
STAR      21594
QSO       18961
Name: count, dtype: int64
```

Persentase:

```
class
GALAXY    59.4
STAR      21.6
QSO       19.0
Name: proportion, dtype: float64 %
```

```
# Cek nilai sentinel/outlier ekstrem pada kolom magnitudo (u, g, r, i, z)
photo_cols = ["u", "g", "r", "i", "z"]
print("Jumlah baris dengan nilai sentinel (< -100) per kolom:")
print((df[photo_cols] < -100).sum())
```

Jumlah baris dengan nilai sentinel (< -100) per kolom:

```
u    1
g    1
r    0
i    0
z    1
dtype: int64
```

```
# Visualisasi sebaran fitur fotometrik terhadap kelas
fig, axes = plt.subplots(2, 3, figsize=(15, 8))
axes = axes.flatten()
for i, col in enumerate(photo_cols + ["redshift"]):
    sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
    axes[i].set_title(f"Distribusi {col} per Kelas")
plt.tight_layout()
plt.show()
```

```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```

```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```

```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```

```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```

```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

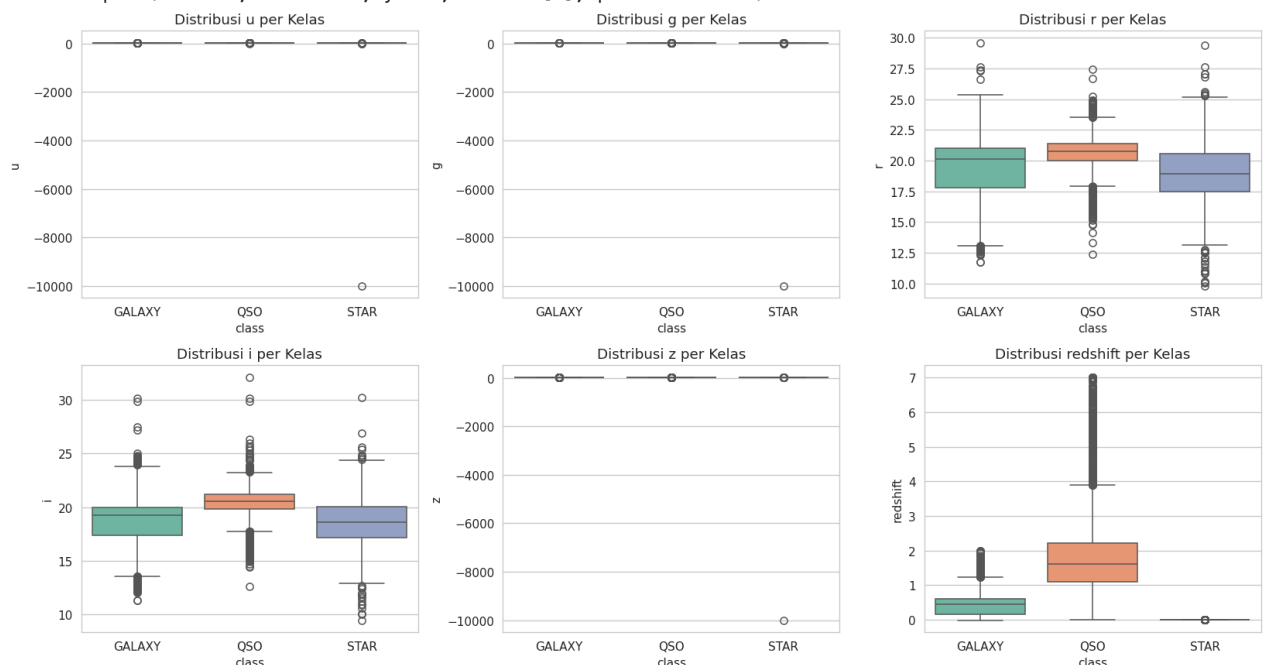
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```

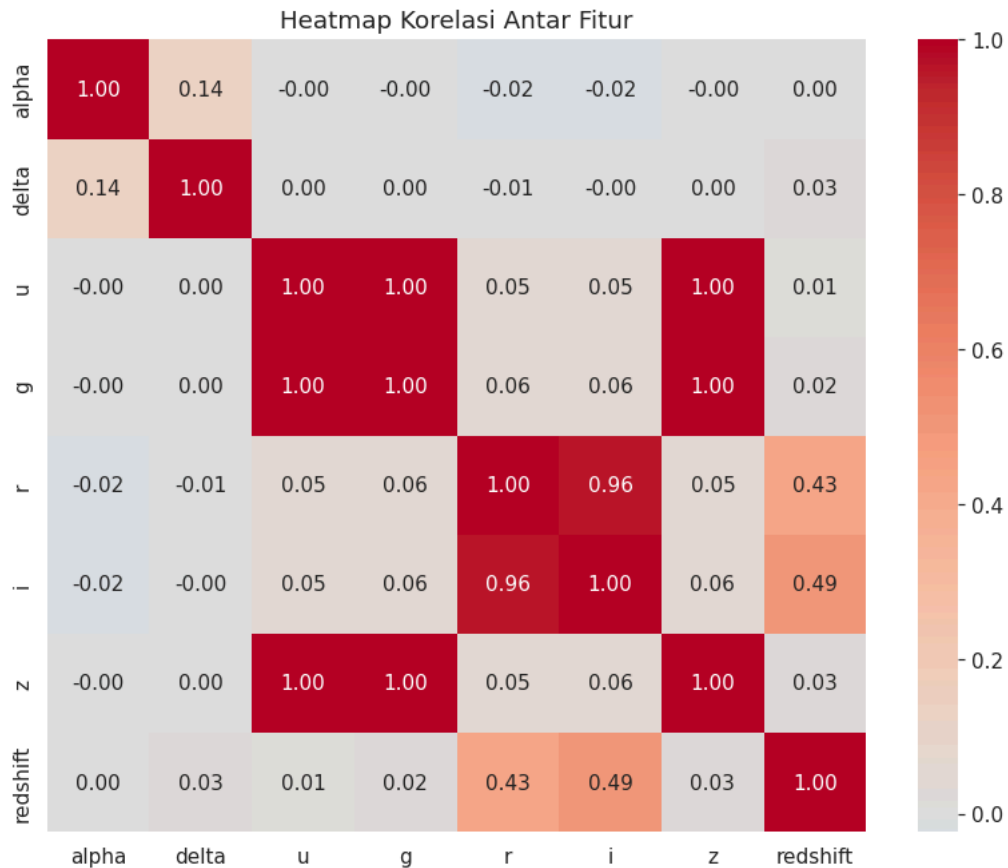
```
/tmp/ipykernel_865/2397598315.py:5: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to a categorical variable with names in the palette.

```
sns.boxplot(data=df, x="class", y=col, ax=axes[i], palette="Set2")
```



```
# Heatmap korelasi
feature_cols_for_corr = ["alpha", "delta", "u", "g", "r", "i", "z", "redshift"]
plt.figure(figsize=(9, 7))
sns.heatmap(df[feature_cols_for_corr].corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Heatmap Korelasi Antar Fitur")
plt.show()
```



3. Data Cleaning & Preprocessing

Langkah-langkah:

1. **Membuang baris dengan nilai sentinel** (-9999) pada kolom fotometrik.
2. **Membuang kolom identifiier/administratif** (`obj_ID`, `run_ID`, `rerun_ID`, `cam_col`, `field_ID`, `spec_obj_ID`, `plate`, `MJD`, `fiber_ID`).
3. **Encoding label** kelas target (`class`) menjadi numerik.
4. **Feature scaling** (standarisasi) sebelum SMOTE diterapkan.

```
photo_cols = ["u", "g", "r", "i", "z"]

# 1. Hapus baris dengan nilai sentinel
before = df.shape[0]
df_clean = df[(df[photo_cols] > -100).all(axis=1)].copy()
print(f"Baris dihapus karena nilai sentinel: {before - df_clean.shape[0]}")

# 2. Hapus kolom identifiier/administratif
drop_cols = ["obj_ID", "run_ID", "rerun_ID", "cam_col", "field_ID",
             "spec_obj_ID", "plate", "MJD", "fiber_ID"]
df_clean = df_clean.drop(columns=drop_cols)

print("Ukuran data setelah cleaning:", df_clean.shape)
df_clean.head()
```

Baris dihapus karena nilai sentinel: 1
Ukuran data setelah cleaning: (99999, 9)

	alpha	delta	u	g	r	i	z	class	redshift
0	135.689107	32.494632	23.87882	22.27530	20.39501	19.16573	18.79371	GALAXY	0.634794
1	144.826101	31.274185	24.77759	22.83188	22.58444	21.16812	21.61427	GALAXY	0.779136
2	142.188790	35.582444	25.26307	22.66389	20.60976	19.34857	18.94827	GALAXY	0.644195
3	338.741038	-0.402828	22.13682	23.77656	21.61162	20.50454	19.25010	GALAXY	0.932346
4	345.282593	21.183866	19.43718	17.58028	16.49747	15.97711	15.54461	GALAXY	0.116123

Next steps: [Generate code with df_clean](#) [New interactive sheet](#)

```
# 3. Encoding label kelas
le = LabelEncoder()
df_clean["class_encoded"] = le.fit_transform(df_clean["class"])

print("Mapping label:")
for i, label in enumerate(le.classes_):
    print(f" {label} -> {i}")

X = df_clean.drop(columns=["class", "class_encoded"])
y = df_clean["class_encoded"]

print("\nFitur yang digunakan:", list(X.columns))

Mapping label:
GALAXY -> 0
QSO -> 1
STAR -> 2

Fitur yang digunakan: ['alpha', 'delta', 'u', 'g', 'r', 'i', 'z', 'redshift']
```

4. Pembagian Data Latih/Uji dan Standardisasi Fitur

Penting: Split dilakukan **sebelum** SMOTE diterapkan. SMOTE hanya akan diaplikasikan pada **data latih** saja (`X_train_scaled`), sehingga data uji tetap mencerminkan distribusi asli dan tidak terjadi **data leakage**.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

print("Jumlah data latih :", X_train.shape[0])
print("Jumlah data uji   :", X_test.shape[0])

# Standardisasi
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nDistribusi kelas pada data latih (sebelum SMOTE):")
unique, counts = np.unique(y_train, return_counts=True)
for u, c in zip(unique, counts):
    print(f" {le.classes_[u]} ({u}): {c}")

Jumlah data latih : 79999
Jumlah data uji   : 20000

Distribusi kelas pada data latih (sebelum SMOTE):
GALAXY (0): 47556
QSO (1): 15169
STAR (2): 17274
```

5. Penerapan SMOTE pada Data Latih

SMOTE (**S**ynthetic **M**inority **O**ver-sampling **T**echnique) bekerja dengan membuat sampel sintetis dari kelas minoritas menggunakan interpolasi antara sampel nyata yang berdekatan (k-nearest neighbors).

Mengapa SMOTE hanya pada data latih?

- Menghindari *data leakage* — data uji harus merepresentasikan distribusi nyata
- Menghindari *overfitting evaluasi* — akurasi pada data sintetis bisa menyesatkan
- Ini adalah praktik standar dalam pipeline ML yang benar

```
smote = SMOTE(random_state=RANDOM_STATE)
X_train_smote, y_train_smote = smote.fit_resample(X_train_scaled, y_train)

print("✅ SMOTE berhasil diterapkan pada data latih")
print(f"Ukuran X_train sebelum SMOTE: {X_train_scaled.shape}")
print(f"Ukuran X_train setelah SMOTE: {X_train_smote.shape}")

print("\nDistribusi kelas pada data latih (setelah SMOTE):")
unique_s, counts_s = np.unique(y_train_smote, return_counts=True)
for u, c in zip(unique_s, counts_s):
    print(f" {le.classes_[u]} ({u}): {c}")

✅ SMOTE berhasil diterapkan pada data latih
Ukuran X_train sebelum SMOTE: (79999, 8)
```

Ukuran X_train setelah SMOTE: (142668, 8)

Distribusi kelas pada data latih (setelah SMOTE):

GALAXY (0): 47556
QSO (1): 47556
STAR (2): 47556

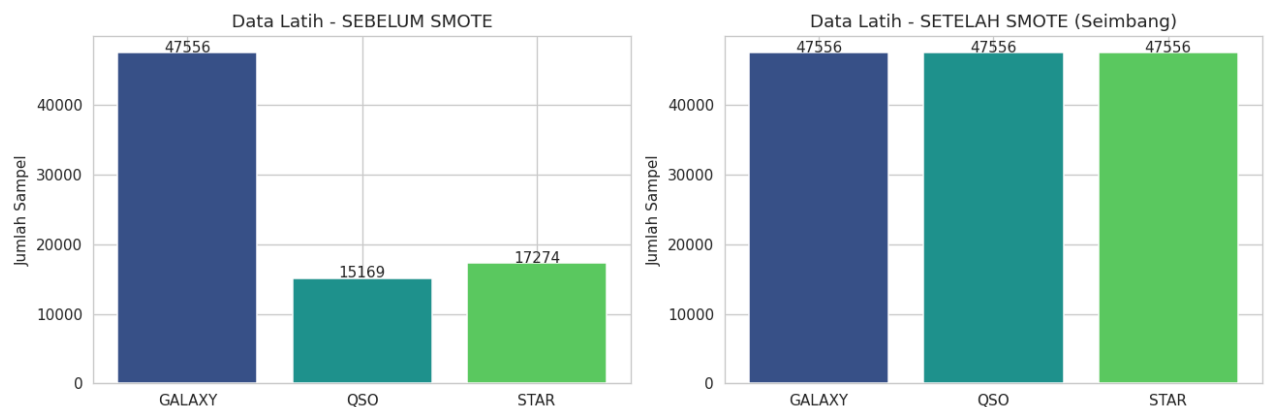
```
# Visualisasi distribusi kelas: sebelum vs sesudah SMOTE
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Sebelum
unique_b, counts_b = np.unique(y_train, return_counts=True)
axes[0].bar([le.classes_[u] for u in unique_b], counts_b, color=sns.color_palette("viridis", 3))
axes[0].set_title("Data Latih - SEBELUM SMOTE")
axes[0].set_ylabel("Jumlah Sampel")
for i, v in enumerate(counts_b):
    axes[0].text(i, v + 100, str(v), ha='center')

# Sesudah
unique_s, counts_s = np.unique(y_train_smote, return_counts=True)
axes[1].bar([le.classes_[u] for u in unique_s], counts_s, color=sns.color_palette("viridis", 3))
axes[1].set_title("Data Latih - SETELAH SMOTE (Seimbang)")
axes[1].set_ylabel("Jumlah Sampel")
for i, v in enumerate(counts_s):
    axes[1].text(i, v + 100, str(v), ha='center')

plt.tight_layout()
plt.show()

print("\nData uji TIDAK disentuh SMOTE - distribusi tetap alami:")
unique_t, counts_t = np.unique(y_test, return_counts=True)
for u, c in zip(unique_t, counts_t):
    print(f" {le.classes_[u]} ({u}): {c}")
```



Data uji TIDAK disentuh SMOTE - distribusi tetap alami:

GALAXY (0): 11889
QSO (1): 3792
STAR (2): 4319

6. Pemodelan: Naïve Bayes, KNN, dan Decision Tree

Semua model dilatih menggunakan `X_train_smote` (data latih yang sudah diseimbangkan), dan dievaluasi pada `X_test_scaled` (data uji dengan distribusi asli).

6.1 Naïve Bayes (GaussianNB)

```
start = time.time()
nb_model = GaussianNB()
nb_model.fit(X_train_smote, y_train_smote)
y_pred_nb = nb_model.predict(X_test_scaled)
nb_time = time.time() - start
print(f"Waktu training Naive Bayes: {nb_time:.4f} detik")
```

Waktu training Naive Bayes: 0.0752 detik

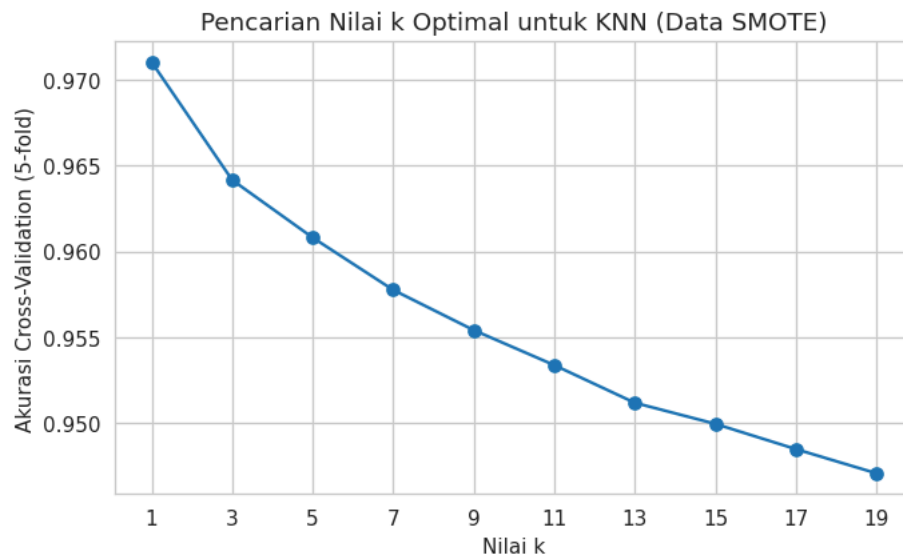
6.2 K-Nearest Neighbors (KNN)

Nilai k optimal dicari menggunakan *elbow method* dengan cross-validation pada data SMOTE.

```
k_range = range(1, 21, 2)
cv_scores = []
for k in k_range:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn_temp, X_train_smote, y_train_smote, cv=5,
                             scoring="accuracy", n_jobs=-1)
    cv_scores.append(scores.mean())

plt.figure(figsize=(7,4))
plt.plot(list(k_range), cv_scores, marker="o")
plt.xlabel("Nilai k")
plt.ylabel("Akurasi Cross-Validation (5-fold)")
plt.title("Pencarian Nilai k Optimal untuk KNN (Data SMOTE)")
plt.xticks(list(k_range))
plt.show()

best_k = list(k_range)[int(np.argmax(cv_scores))]
print(f"Nilai k terbaik: {best_k} (akurasi CV: {max(cv_scores):.4f})")
```



Nilai k terbaik: 1 (akurasi CV: 0.9710)

```
start = time.time()
knn_model = KNeighborsClassifier(n_neighbors=best_k)
knn_model.fit(X_train_smote, y_train_smote)
y_pred_knn = knn_model.predict(X_test_scaled)
knn_time = time.time() - start
print(f"Waktu training KNN (k={best_k}): {knn_time:.4f} detik")
```

Waktu training KNN (k=1): 1.6204 detik

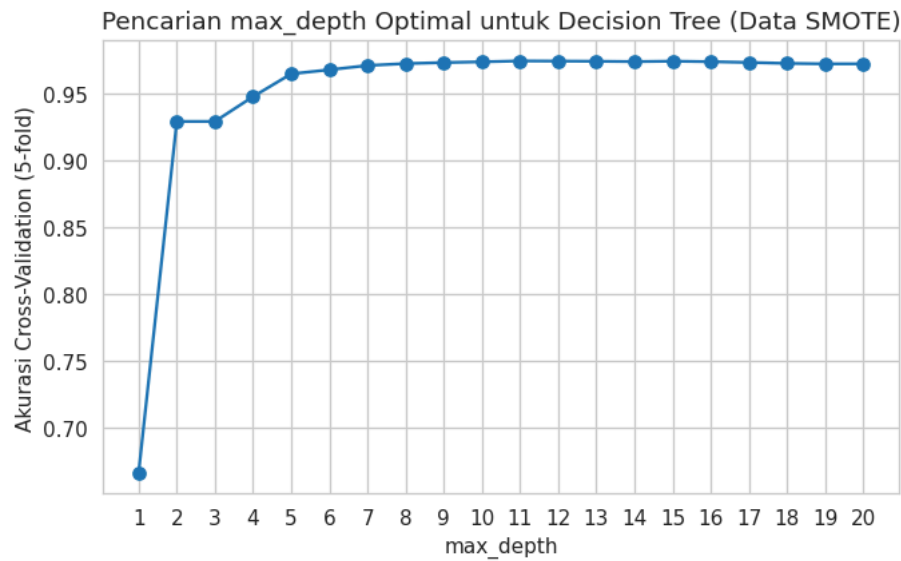
6.3 Decision Tree

```
depth_range = range(1, 21)
cv_scores_dt = []
for d in depth_range:
    dt_temp = DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=d)
    scores = cross_val_score(dt_temp, X_train_smote, y_train_smote, cv=5,
                             scoring="accuracy", n_jobs=-1)
    cv_scores_dt.append(scores.mean())

plt.figure(figsize=(7,4))
plt.plot(list(depth_range), cv_scores_dt, marker="o")
plt.xlabel("max_depth")
plt.ylabel("Akurasi Cross-Validation (5-fold)")
plt.title("Pencarian max_depth Optimal untuk Decision Tree (Data SMOTE)")
plt.xticks(list(depth_range))
plt.show()
```



```
best_depth = list(depth_range)[int(np.argmax(cv_scores_dt))]  
print(f"max_depth terbaik: {best_depth} (akurasi CV: {max(cv_scores_dt):.4f})")
```

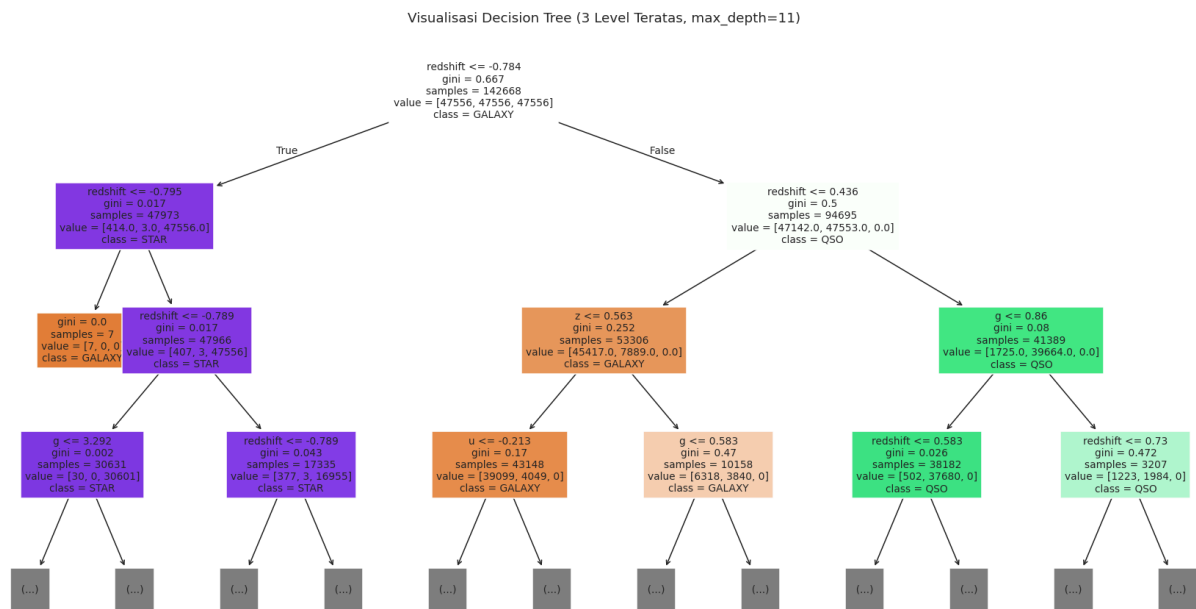


max_depth terbaik: 11 (akurasi CV: 0.9745)

```
start = time.time()  
dt_model = DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=best_depth)  
dt_model.fit(X_train_smote, y_train_smote)  
y_pred_dt = dt_model.predict(X_test_scaled)  
dt_time = time.time() - start  
print(f"Waktu training Decision Tree: {dt_time:.4f} detik")
```

Waktu training Decision Tree: 4.2755 detik

```
# Visualisasi pohon keputusan (3 level terbatas)  
plt.figure(figsize=(20, 10))  
plot_tree(dt_model, max_depth=3, feature_names=X.columns, class_names=le.classes_,  
          filled=True, fontsize=9)  
plt.title(f"Visualisasi Decision Tree (3 Level Teratas, max_depth={best_depth})")  
plt.show()
```



```

# Feature Importance dari Decision Tree
importances = dt_model.feature_importances_
feat_importance_df = pd.DataFrame({
    "Fitur": X.columns,
    "Importance": importances
}).sort_values("Importance", ascending=False).reset_index(drop=True)

print(feat_importance_df)

plt.figure(figsize=(8, 5))
sns.barplot(data=feat_importance_df, x="Importance", y="Fitur", palette="crest")
plt.title(f"Feature Importance Decision Tree dengan SMOTE (max_depth={best_depth})")
plt.xlabel("Tingkat Kepentingan (Gini Importance)")
plt.ylabel("Fitur")
plt.tight_layout()
plt.show()

```

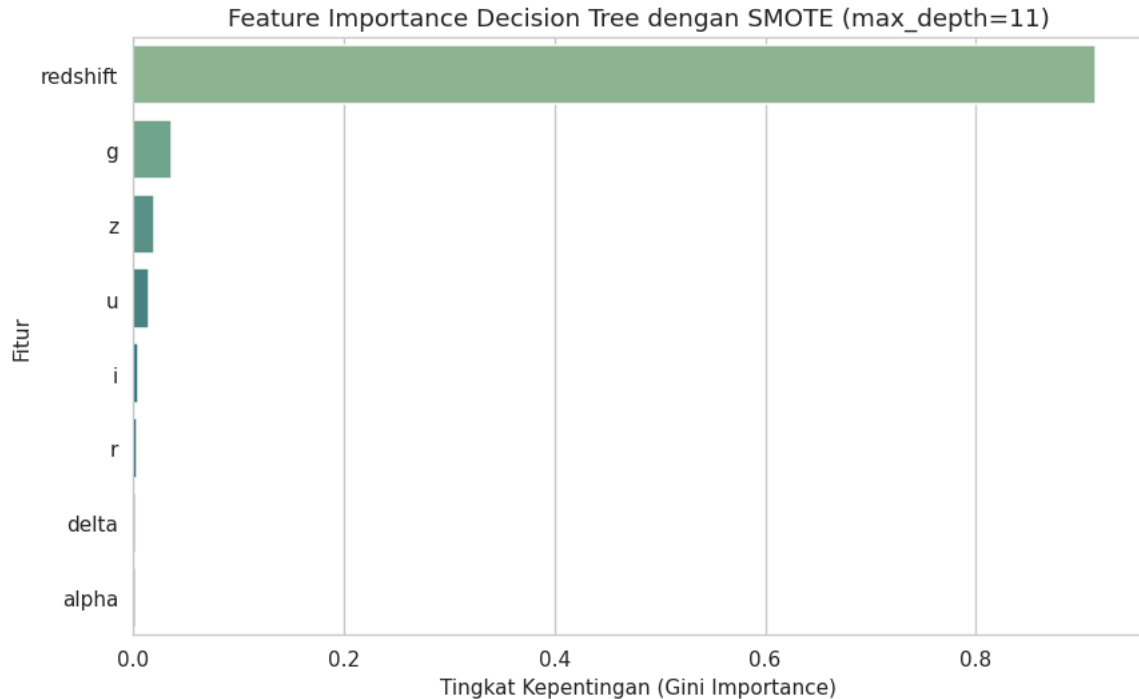
```

Fitur Importance
0  redshift  0.913555
1      g    0.036715
2      z    0.020352
3      u    0.015349
4      i    0.005134
5      r    0.003706
6  delta    0.002602
7  alpha    0.002586
/tmp/ipykernel_865/1069240377.py:11: FutureWarning:

```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to

```
sns.barplot(data=feat_importance_df, x="Importance", y="Fitur", palette="crest")
```



6.4 Validasi Stabilitas Model dengan Cross-Validation (Data SMOTE)

```

cv_strategy = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

models_for_cv = {
    "Naive Bayes": GaussianNB(),
    f"KNN (k={best_k})": KNeighborsClassifier(n_neighbors=best_k),
    "Decision Tree": DecisionTreeClassifier(random_state=RANDOM_STATE, max_depth=best_depth),
}

cv_results = []
for name, model in models_for_cv.items():
    scores = cross_val_score(model, X_train_smote, y_train_smote,
                             cv=cv_strategy, scoring="accuracy", n_jobs=-1)

    cv_results.append({
        "Model": name,
        "Mean Accuracy (CV)": scores.mean(),
        "Std Dev (CV)": scores.std(),
        "Scores per Fold": np.round(scores, 4)
    })
    print(f"{name}: {scores.mean():.4f} ± {scores.std():.4f} | fold scores: {np.round(scores, 4)}")

cv_df = pd.DataFrame(cv_results).set_index("Model")
cv_df[["Mean Accuracy (CV)", "Std Dev (CV)"]].round(4)

```

Naive Bayes: 0.9213 ± 0.0008 | fold scores: [0.9203 0.9213 0.9212 0.9212 0.9227]
 KNN (k=1): 0.9702 ± 0.0010 | fold scores: [0.9712 0.9696 0.971 0.9686 0.9706]
 Decision Tree: 0.9744 ± 0.0010 | fold scores: [0.973 0.9751 0.9754 0.9752 0.9735]

Mean Accuracy (CV) Std Dev (CV)

Model		
Naive Bayes	0.9213	0.0008
KNN (k=1)	0.9702	0.0010
Decision Tree	0.9744	0.0010

7. Evaluasi Model pada Data Uji (Distribusi Alami)

Data uji menggunakan distribusi **asli** (tidak di-SMOTE), sehingga evaluasi ini mencerminkan performa model pada skenario nyata.

```
def evaluate(y_true, y_pred, name):
    return {
        "Model": name,
        "Accuracy": accuracy_score(y_true, y_pred),
        "Precision (macro)": precision_score(y_true, y_pred, average="macro", zero_division=0),
        "Recall (macro)": recall_score(y_true, y_pred, average="macro", zero_division=0),
        "F1-Score (macro)": f1_score(y_true, y_pred, average="macro", zero_division=0),
    }

results = pd.DataFrame([
    evaluate(y_test, y_pred_nb, "Naive Bayes") | {"Waktu Training (s)": round(nb_time, 4)},
    evaluate(y_test, y_pred_knn, f"KNN (k={best_k})") | {"Waktu Training (s)": round(knn_time, 4)},
    evaluate(y_test, y_pred_dt, "Decision Tree") | {"Waktu Training (s)": round(dt_time, 4)},
])

results = results.set_index("Model").round(4)
print("=== Ringkasan Performa (dengan SMOTE) ===")
results
```

=== Ringkasan Performa (dengan SMOTE) ===

	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)	Waktu Training (s)
Model					
Naive Bayes	0.8998	0.8785	0.9210	0.8937	0.0752
KNN (k=1)	0.9310	0.9203	0.9269	0.9236	1.6204
Decision Tree	0.9730	0.9661	0.9721	0.9690	4.2755

Next steps: [Generate code with results](#)

[New interactive sheet](#)

```
# Classification report lengkap per model
for name, y_pred in [("Naive Bayes", y_pred_nb),
                    (f"KNN (k={best_k})", y_pred_knn),
                    ("Decision Tree", y_pred_dt)]:
    print(f"=== {name} ===")
    print(classification_report(y_test, y_pred, target_names=le.classes_, zero_division=0))
    print()
```

```
=== Naive Bayes ===
      precision    recall  f1-score   support

   GALAXY       0.97       0.86       0.91       11889
      QSO       0.69       0.91       0.78       3792
      STAR       0.98       0.99       0.98       4319
```

```
   accuracy
macro avg       0.88       0.92       0.89       20000
weighted avg       0.92       0.90       0.90       20000
```

```
=== KNN (k=1) ===
      precision    recall  f1-score   support

   GALAXY       0.95       0.94       0.94       11889
      QSO       0.90       0.92       0.91       3792
      STAR       0.91       0.92       0.91       4319
```

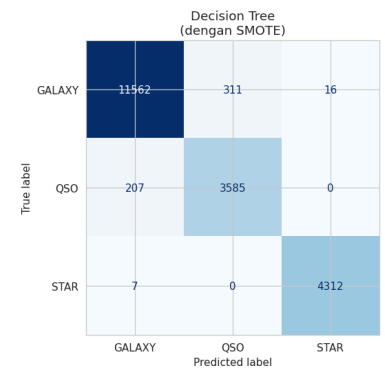
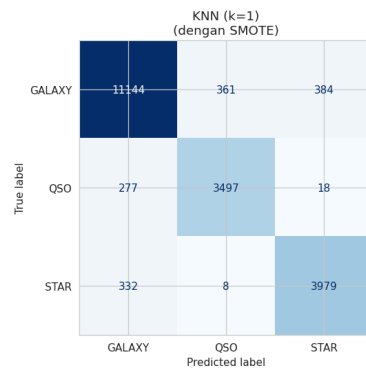
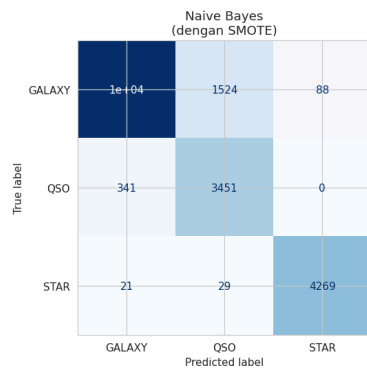
```
   accuracy
macro avg       0.92       0.93       0.92       20000
weighted avg       0.93       0.93       0.93       20000
```

```
=== Decision Tree ===
      precision    recall  f1-score   support

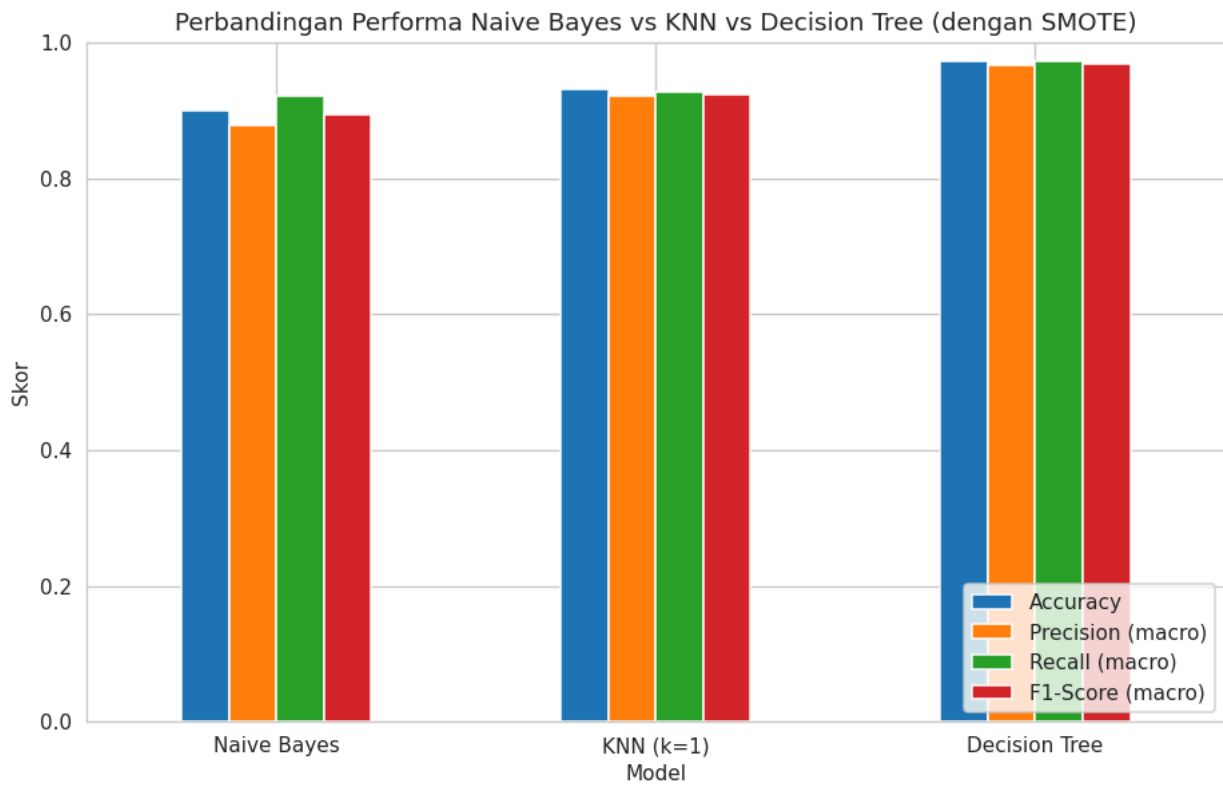
   GALAXY       0.98       0.97       0.98       11889
      QSO       0.92       0.95       0.93       3792
      STAR       1.00       1.00       1.00       4319
```

```
   accuracy
macro avg       0.97       0.97       0.97       20000
weighted avg       0.97       0.97       0.97       20000
```

```
# Confusion matrix untuk ketiga model
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for ax, (name, y_pred) in zip(axes, [{"Naive Bayes", y_pred_nb},
                                     (f"KNN (k={best_k})", y_pred_knn),
                                     ("Decision Tree", y_pred_dt)]):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    ax.set_title(f"{name}\n(dengan SMOTE)")
plt.tight_layout()
plt.show()
```



```
# Grafik perbandingan metrik antar model
metrics_to_plot = ["Accuracy", "Precision (macro)", "Recall (macro)", "F1-Score (macro)"]
results[metrics_to_plot].plot(kind="bar", figsize=(10, 6), rot=0)
plt.title("Perbandingan Performa Naive Bayes vs KNN vs Decision Tree (dengan SMOTE)")
plt.ylabel("Skor")
plt.ylim(0, 1)
plt.legend(loc="lower right")
plt.show()
```



```
# Tabel ringkasan final
print("=== Ranking Model Berdasarkan F1-Score (dengan SMOTE) ===")
results.sort_values("F1-Score (macro)", ascending=False)
```

```
=== Ranking Model Berdasarkan F1-Score (dengan SMOTE) ===
```

	Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)	Waktu Training (s)
Model					
Decision Tree	0.9730	0.9661	0.9721	0.9690	4.2755
KNN (k=1)	0.9310	0.9203	0.9269	0.9236	1.6204
Naive Bayes	0.8998	0.8785	0.9210	0.8937	0.0752

8. Diskusi: Pengaruh SMOTE terhadap Performa Model

Tabel di bawah ini dapat diisi secara manual setelah menjalankan kedua notebook (dengan dan tanpa SMOTE), untuk membandingkan pengaruhnya.

Model	F1 Tanpa SMOTE	F1 Dengan SMOTE	Selisih
Naive Bayes	-	-	-
KNN	-	-	-
Decision Tree	-	-	-

Interpretasi yang diharapkan:

- Jika F1-Score (macro) **meningkat** dengan SMOTE → model lebih adil dalam mengenali semua kelas
- Jika **akurasi turun sedikit** dengan SMOTE → wajar, karena model tidak lagi "bias" ke kelas mayoritas (GALAXY)
- Recall kelas minoritas** (STAR, QSO) biasanya meningkat signifikan dengan SMOTE

Catatan filosofis: Data uji tidak di-SMOTE sengaja — kita ingin tahu, "setelah dilatih dengan data seimbang, seberapa baik model menghadapi dunia nyata yang tidak seimbang?" Inilah esensi mengapa evaluasi tetap pada distribusi asli.

.
.
.
.
.
.
.
.
.
.
.